SMOLR: A RISC-V-Native Minsize Linker and Runtime Import System

Status

Draft design document for discussion with toolchain/runtime specialists.

Working name

SMOLR

Possible expansions:

- **SMOL for RISC-V**
- **Small Minsize-Oriented Linker for RISC-V**
- **Shoddy Minsize-Oriented Linker, Reimagined**
- **Small, Minsize-Oriented Linker, Relaxation-aware**

Preferred project identity:

> **SMOLR is a RISC-V-native sizecoding linker and runtime import system for tiny Linux demos.**

The name should be allowed to stay slightly ridiculous. Demoscene tools benefit from having a gremlin-shaped silhouette.

---

1. Purpose

SMOLR exists to make extremely small dynamically linked Linux executables for RISC-V, especially RV64GC Linux systems such as Ubuntu Desktop on RISC-V boards.

It is inspired by SMOL, UPX, Crinkler, dnload, and Linux sizecoding practice, but should not begin as a blind port of any existing tool. RISC-V has different instruction encoding, relaxation behavior, compressed instructions, relocation pairs, and ABI details. SMOLR should exploit those deliberately.

The first serious target is not general application packaging. The first serious target is demoscene output:

- tiny graphical/audio/text demos,
- dynamically linked against system libraries,
- built for RV64GC Linux,
- possibly compressed afterward,
- with strict control over bytes.

---

2. Design philosophy

2.1 Build the smallest useful thing first

Do not start with a complete linker.

Start with:

> One RISC-V64 Linux ELF executable, handmade/minimized, dynamically resolving and calling one external libc function.

Everything else follows from that.

2.2 Linker first, compressor second

UPX compresses an already linked executable. SMOL-like tools reduce the executable before compression by avoiding normal dynamic-linking metadata overhead.

SMOLR should therefore be designed as:

1. **Minsize linker/runtime import system**
2. **RISC-V code-shaping layer**
3. **Optional compression/packer layer**

Compression is important, but it should not hide bad linking structure. First make the ELF and imports tiny. Then compress.

2.3 RISC-V-native, not x86-in-RISC-V clothing

A direct SMOL port may drag along x86 assumptions:

- NASM syntax,
- x86 PLT/GOT patterns,
- x86 relocation types,
- x86 dynamic-loader shortcuts,
- x86 code-size habits.

SMOLR should instead use RISC-V's own strengths:

- RVC compressed instructions,
- AUIPC/JALR call patterns,
- linker relaxation,
- compact register choices,
- RV64GC ABI conventions,
- predictable load/store architecture,
- careful PC-relative data access.

2.4 Correctness before size heroics

The first version may be larger than ideal. That is acceptable.

Milestone order:

1. Runnable.
2. Correct.
3. Reproducible.
4. Measured.
5. Smaller.
6. Smaller still.

The first working resolver can be boring. The second can be sharp. The third can start stealing spoons from the byte cupboard.

---

3. Target platform

3.1 Initial target

SMOLR should be **RVA23-first**, not merely RV64GC-first.

Primary target:

- Profile: **RVA23U64**
- Architecture class: **64-bit RISC-V application processor profile**
- ABI: **LP64D**
- OS: **Linux**
- Userland: **glibc first**
- Binary type: **dynamically linked ELF64 executable**
- Toolchain: Clang/LLVM or GCC/binutils with explicit RVA23/profile support
- Test runtime: `qemu-riscv64`, then real RISC-V Ubuntu/Desktop board

Minimum fallback target:

- Architecture: **RV64GCV + scalar bitmanip B**
- Required fallback extensions at minimum:
- `V`
- `C`
- scalar bitmanip: `B`, or explicitly `Zba_Zbb_Zbs` and possibly `Zbc`
- preferably `Zcb`, `Zicond`, `Zfa`, `Zfhmin`, `Zvbb`

The design should treat generic `rv64gc` as a debug/bootstrap target only, not as the intended performance/size target.

3.1.1 Why RVA23 matters

RVA23 gives SMOLR a much stronger baseline:

- vector instructions are guaranteed,
- scalar bit manipulation is guaranteed,
- vector basic bit manipulation is guaranteed,
- additional compressed instructions are available,
- conditional-zero instructions are available,
- cache block and non-temporal hints can be assumed where useful,
- profile-level guarantees reduce hardware fragmentation.

This changes the project from:

```
make a tiny generic RISC-V ELF
```

into:

```
make the best tiny Linux executable toolchain for modern RISC-V application
processors
```

3.1.2 Recommended architecture levels

SMOLR should expose explicit target levels:

```
smolr-rv64gc
    Bootstrap/debug target only. No vector assumptions.

smolr-rv64gcv-b
    Practical fallback target. Requires V and scalar bitmanip.

smolr-rva23u64
    Primary target. Assumes the full RVA23U64 mandatory user-mode profile.

smolr-rva23u64+crypto
    Optional future target for vector crypto experiments.
```

The first production-quality backend should be `smolr-rva23u64`.

3.2 Not initial targets

Explicitly out of scope for phase 1:

- RV32
- musl
- static linking

- C++
- exceptions
- RTTI
- constructors/destructors
- TLS
- external global data symbols
- IFUNC
- full Cairo/Pango demos
- complete POSIX compatibility
- arbitrary third-party binaries

SMOLR is not a general packer. It is a precision tool for tiny controlled programs.

---

4. Quick RISC-V control-flow primer

This section exists because SMOLR must be relaxation-aware.

4.1 `jal`

Conceptually:

```
jal rd, target
```

Effect:

```
rd = pc + 4
target_pc = pc + signed_offset
pc = target_pc
```

`jal` is PC-relative and can reach a limited range. On RV64, the normal direct call pseudo-instruction may become `jal ra, target` if the target is close enough.

Important:

- `jal x0, target` is an unconditional jump with no return address.
- `j target` is a pseudo-instruction for `jal x0, target`.
- `jal ra, target` stores return address in `ra`.

4.2 `jalr`

Conceptually:

```
jalr rd, imm(rs1)
```

Effect:

```
rd = pc + 4
target = (rs1 + sign_extend(imm12)) & ~1
pc = target
```

Important:

- `ret` is `jalr x0, 0(ra)`.
- `jr rs1` is `jalr x0, 0(rs1)`.
- `jalr` is useful when the destination is in a register, such as an imported function pointer.

4.3 `auipc`

Conceptually:

```
auipc rd, imm20
```

Effect:

```
rd = pc_of_auipc + sign_extend(imm20 << 12)
```

Important correction:

`auipc` does **not** change `pc`. It writes a PC-relative address base into `rd`.

Also, it does **not** zero the low 12 bits of `pc`. The immediate contributes zeros in its low 12 bits, but the current PC is added afterward. The resulting register value may have nonzero low bits inherited from the instruction address.

4.4 Why `auipc` plus `jalr` exists

A call target may be too far for `jal`.

So the assembler/linker can represent a call as:

```
auipc ra, %pcrel_hi(target)
jalr  ra, %pcrel_lo(target)(ra)
```

This reaches a much larger PC-relative range.

If the linker later sees the target is close enough, it can replace the pair with:

```
jal ra, target
```

That is **function call relaxation**.

4.5 Linker relaxation in one sentence

Linker relaxation is the linker replacing conservative, longer instruction sequences with shorter equivalent ones after final addresses are known.

For SMOLR this matters because the final code size is not known merely by counting assembler source lines. The linker may shorten calls, branches, address computations, and sometimes compressed forms.

4.6 SMOLR rule

SMOLR must not fight relaxation blindly.

The rule:

> Emit normal relocation-marked RISC-V sequences first, let the linker relax them, then measure and optimize.

Hand-patching byte offsets too early is dangerous. RISC-V relaxation depends on relocation records.

---

5. What SMOLR should do better than UPX

UPX compresses a normal executable after it has already been linked.

SMOLR should reduce the executable before compression by avoiding or replacing bulky ELF/dynamic-linking structures.

SMOLR should therefore focus on:

- tiny ELF headers,
- minimal program headers,
- compact dynamic table,
- compact `DT_NEEDED` list,
- compact imported-symbol table,
- runtime symbol resolution,
- tiny import stubs,
- RVC-friendly runtime code,
- linker-relaxation-aware layout,
- optional cooperation with a later packer.

UPX remains useful as:

- baseline,
- fallback,
- post-link packer,
- comparison target.

SMOLR should not try to become UPX first. It should make a much smaller input for any later packer.

---

6. What SMOLR should do better than SMOL

Existing SMOL is x86/i386 and x86_64 oriented. SMOLR should be RISC-V-native.

Potential advantages over a direct SMOL port:

- cleaner GNU assembler backend,
- RV64GC-first ABI design,
- RVC-aware register allocation in runtime code,
- relaxation-aware call/data-access sequences,
- import stubs designed for RISC-V from the start,
- benchmark matrix including `normal ld`, `UPX`, `SMOLR`, and `SMOLR + packer`,
- no inherited NASM structure,
- clearer separation between linker, runtime resolver, and optional compression.

A good final result could later be upstreamed into SMOL or kept as a separate tool. That decision should wait until after a working RV64GC proof of concept.

---

7. Core architecture

7.1 Build pipeline

Proposed pipeline:

```
input objects/libs
    ↓
SMOLR scanner
    ↓
relocation and import analysis
    ↓
minimal ELF layout planner
    ↓
RISC-V runtime/import stub generator
    ↓
GNU ld or custom layout link step
    ↓
optional truncation/section stripping
    ↓
optional packer/compressor
    ↓
final tiny executable
```

7.2 Major components

A. Frontend scanner

Responsibilities:

- inspect input ELF objects,
- verify RV64GC/LP64D compatibility,
- gather undefined global function symbols,
- gather relocation types,
- reject unsupported features clearly,
- identify required shared libraries,
- build import list.

B. Library/symbol resolver at build time

Responsibilities:

- search compiler/library paths,
- identify matching RISC-V ELF shared libraries,
- inspect exported symbols,
- map each needed symbol to a library,
- construct `DT_NEEDED` order,
- detect ambiguous definitions.

C. ELF layout planner

Responsibilities:

- construct minimal ELF64 header,
- construct program headers,
- define loadable segments,
- construct dynamic table,
- decide RX/RW split or single RWX segment,
- place runtime resolver,
- place import table,
- place user code/data.

D. RISC-V runtime resolver

Responsibilities:

- locate dynamic linker/library state,
- walk loaded shared libraries,
- locate symbol/string/hash tables,
- find imported symbols,
- write resolved addresses into SMOLR import table,
- transfer control to user `_start`.

E. RISC-V import stubs

Responsibilities:

For each imported function, provide a callable symbol such as:

```
.globl puts
puts:
    auipc t0, %pcrel_hi(_smolr_sym_puts)
    ld    t0, %pcrel_lo(.Lputs_sym)(t0)
    jr    t0
```

Exact syntax must be validated with GNU as and binutils.

F. Optional post-link packer interface

Responsibilities:

- feed final output into UPX or custom packer,
- measure size before/after,
- support reproducible benchmark tables.

---

8. Import symbol representation

SMOLR should not carry full dynamic symbol names in the normal ELF way unless needed.

Recommended starting point:

```
_imports:
    .quad hash(puts)
    .quad hash(write)
    .quad hash(cairo_create)
    .quad 0
```

At runtime, resolver overwrites each hash slot with the actual resolved function address.

Alternative structure:

```
struct smolr_import {
    uint32_t hash;
    uint16_t lib_index;
    uint16_t flags;
    uint64_t resolved_address;
}
```

This is larger, but easier to debug.

Recommended plan:

- Phase 1: larger debug-friendly table.
- Phase 2: compact hash-address overlay table.
- Phase 3: specialized per-library tables if that shrinks resolver logic.

---

9. Hash strategy

Candidate hash functions:

- DJB2, simple and small.
- BSD-style 16-bit hash, smaller tables but more collisions.
- CRC32C, interesting if hardware extension exists, but not baseline RV64GC.
- GNU hash reuse, possibly avoids custom hash but may increase runtime complexity.

Initial recommendation:

```
Use DJB2 first.
```

Reason:

- simple implementation,
- portable,
- already familiar from SMOL-like tools,
- good enough for initial symbol lists.

Later experiments:

- 16-bit hash with collision fallback,
- per-library sorted symbol hashes,
- direct name comparison for very small import sets,
- perfect hash generated at build time for known imports.

---

10. Relocation support

10.1 Initial supported input

Support only external function calls generated by controlled compiler flags.

Recommended first supported relocation family:

- `R_RISCV_CALL_PLT`
- `R_RISCV_CALL`

- associated `R_RISCV_RELAX`

The exact list must be confirmed empirically using:

```
riscv64-linux-gnu-readelf -rW file.o
riscv64-linux-gnu-objdump -dr file.o
```

10.2 Unsupported at first

Reject cleanly:

- TLS relocations,
- external data objects,
- complex GOT data references,
- IFUNC,
- copy relocations,
- C++ exception/unwind machinery,
- constructors/destructors.

The error messages should teach the user what compiler flag or source change caused the problem.

Example:

```
error: external data symbol 'errno' requires unsupported RISC-V GOT/data
relocation.
try avoiding libc errno access or use raw syscalls for this path.
```

---

11. RISC-V code-size and performance strategy

11.0 RVA23-first optimization policy

SMOLR should optimize for the profile actually desired for ENO-class demos:

```
RVA23U64 first.
RV64GCV+B fallback second.
RV64GC generic compatibility last.
```

The runtime resolver may remain scalar if scalar code is smaller, but SMOLR should assume that generated demo code, decompression filters, byte shufflers, hash scanners, transform kernels, text/bitmap processing, and import-table processing may use vector and bitmanip instructions.

The project should avoid a false economy: saving 30 bytes in the resolver is pointless if it prevents using RVA23 features that save hundreds of bytes or many milliseconds elsewhere.

11.1 Use RVC and Zc deliberately

Runtime resolver and stubs should be written with compressed-instruction density in mind.

RVA23 includes the classic compressed baseline plus additional compressed instructions such as Zcb and compressed may-be-operations. SMOLR should therefore measure not only `C`, but also profile-level compressed instruction opportunities.

Guidelines:

- Prefer registers that compress well where possible.
- Prefer `sp`-relative saves/loads with compressible offsets.
- Use `c.li`, `c.mv`, `c.addi`, `c.ldsp`, `c.sdsp`, `c.jr`, `c.j`, `c.beqz`, `c.bnez` where the assembler can emit them.
- Prefer Zcb forms when they replace 32-bit load/extend/multiply/sign-extension sequences.
- Keep small hot loops within short branch ranges.
- Avoid wide temporaries unless they save more elsewhere.

11.2 Use scalar bitmanip where it shrinks code

RVA23 includes scalar bit manipulation. SMOLR should deliberately test whether scalar bitmanip reduces code size or runtime size in:

- symbol hashing,
- string scanning,
- byte/word packing,
- decompressor loops,
- bitmap/A8 mask processing,
- import table scanning,
- small checksum/hash functions,
- branchless bit tricks.

Candidate useful instruction families:

- `Zbb`: rotates, count leading/trailing zeros, population count, byte-reversal style helpers,
- `Zba`: address-generation helpers,
- `Zbs`: single-bit operations,
- `Zbc`: carryless multiply, mostly for CRC/hash experiments where available or required.

Do not assume bitmanip always shrinks code. Measure each kernel.

11.3 Use RVV where it wins at system level

RVV should not be forced into scalar save/restore or tiny call glue. That usually grows code.

RVV is promising for:

- decompression filters,
- RISC-V instruction stream preprocessing before compression,
- byte shuffling,

- fast symbol-name/hash scanning if the resolver becomes large enough,
- A8 alpha mask transforms,
- text surface post-processing,
- wavelet/chirplet/quadrature kernels,
- audio block processing,
- particle/state updates,
- SPINE event expansion.

Design rule:

```
Use scalar/RVC for tiny control glue.
Use RVV for data-parallel kernels.
```

Important RVV design notes:

- VLEN is implementation-defined.
- Code must be vector-length agnostic unless a benchmark deliberately targets a known board.
- Use `vsetvli` / `vsetivli` patterns cleanly.
- Avoid assuming fixed vector register width.
- Keep vector kernels separately measurable.

11.4 Let relaxation work

Do not manually force everything into non-relaxable forms.

Test both:

```
-mrelax
-mno-relax
```

A correct SMOLR must work with either. A best-size SMOLR should normally prefer relaxation enabled.

11.5 Measure actual emitted bytes

Never trust source-level instruction counts.

Required tooling:

```
riscv64-linux-gnu-objdump -dr final.elf
riscv64-linux-gnu-readelf -h -l -d -r -s final.elf
wc -c final.elf
```

The build should emit a size report:

```
headers:       N bytes
runtime:       N bytes
```

```
import table:  N bytes
stubs:         N bytes
user text:     N bytes
user rodata:   N bytes
packed total:  N bytes
```

---

12. ELF design

12.1 Header goals

SMOLR should generate the minimum ELF64/RISC-V program accepted by Linux and the dynamic loader.

Required areas:

- ELF header,
- program headers,
- at least one `PT_LOAD`,
- `PT_DYNAMIC`,
- possibly `PT_INTERP`,
- dynamic entries for needed libraries,
- string table for library names,
- runtime code and import table.

12.2 RISC-V ELF flags

The generated ELF header must use valid RISC-V `e_flags`, especially ABI-related flags for LP64D.

Task:

- Compile a known-good RV64GC/LP64D dynamic executable.
- Inspect `readelf -h` output.
- Reproduce only the required flags.

12.3 Section headers

Final executable should not need section headers.

Goal:

```
e_shoff = 0
e_shnum = 0
e_shstrndx = 0
```

If tools complain but Linux runs it, that is acceptable for release builds. Debug builds may keep sections.

12.4 NX policy

Two modes:

1. Unsafe/minimal single RWX load segment.
2. Safer split RX/RW load segments.

For demoscene competitions, rules and runtime environment decide which mode is acceptable.

SMOLR should support both, but phase 1 may start with the easier one.

---

13. Runtime resolver design

13.1 Correctness-first resolver

First resolver should be readable and robust, even if not smallest.

Pseudo-flow:

```
_smolr_start:
    save original sp if needed
    find link_map root
    for each import hash:
        for each loaded library in DT_NEEDED order:
            find dynamic section
            find STRTAB, SYMTAB, GNU_HASH or SYSV HASH
            search exported symbols
            if hash matches:
                resolved = l_addr + st_value
                write resolved address into import slot
                break
    set a0 or sp contract for user _start
    jump to user _start
```

13.2 Link-map access strategies

Candidate strategies:

1. Use `DT_DEBUG` if available.
2. Use startup/dynamic-linker state assumptions.
3. Use an existing dnload-like approach.

Recommendation:

```
Start with the most understandable strategy, even if larger.
```

Then add smaller glibc-specific shortcuts after tests pass.

13.3 IFUNC

Do not support IFUNC in phase 1.

Later, if common glibc symbols require it, add optional IFUNC resolver support.

---

14. External API and user model

14.1 Input requirements

User code should be compiled with controlled flags.

Example starting flags:

```
riscv64-linux-gnu-gcc
  -march=rv64gc
  -mabi=lp64d
  -Os
  -fno-stack-protector
  -fno-unwind-tables
  -fno-asynchronous-unwind-tables
  -ffunction-sections
  -fdata-sections
  -nostartfiles
  -c demo.c -o demo.o
```

For assembly:

```
riscv64-linux-gnu-as -march=rv64gc demo.s -o demo.o
```

14.2 User entry point

Require:

```
.section .text.startup._start,"ax"
.globl _start
_start:
    ...
```

SMOLR runtime eventually jumps into user `_start`.

Possible startup contract:

```
a0 = original stack pointer
sp = original or aligned stack pointer, depending mode
```

This must be documented and tested.

---

15. Benchmark suite

15.0 Target variants to benchmark

Every serious benchmark should be tested against at least:

```
A. rv64gc
B. rv64gcv_zba_zbb_zbs, plus zbc if available
C. rva23u64
D. rva23u64 plus optional vector crypto, later
```

If the toolchain supports profile names directly, test `-march=rva23u64`. If not, use the explicit expanded ISA string reported by the toolchain for the profile.

Benchmarking must separate:

- code size,
- packed size,
- runtime speed,
- startup overhead,
- vector register use overhead,
- portability across boards.

15.1 Required benchmark programs

1. Raw syscall hello.
2. Dynamic `puts` hello.
3. Dynamic `write` hello.
4. `libm` function call.
5. Minimal framebuffer or SDL/OpenGL/EGL call.
6. Cairo image surface creation.
7. Cairo A8 text render.
8. Cairo plus Pango text render.

15.2 Comparison matrix

For each benchmark:

```
A. normal GNU ld, stripped
B. normal GNU ld + UPX --best
```

```
C. SMOLR uncompressed
D. SMOLR + UPX
E. SMOLR + custom packer, if available
```

Metrics:

- final file size,
- startup success under qemu-riscv64,
- startup success on real hardware,
- number of imports,
- runtime resolver size,
- stub size per import,
- compressed ratio,
- reproducibility.

15.3 Definition of top in class

SMOLR is top in class if it consistently produces the smallest working RV64GC Linux dynamic executables in the benchmark suite, while remaining usable enough for real demo development.

Target claims must be measurement-based, not vibes-based.

---

16. Work plan

Phase 0: decision and setup

Duration: 2-4 days.

Deliverables:

- choose repository structure,
- choose license strategy,
- set up cross-toolchain container,
- set up qemu-riscv64 tests,
- collect current SMOL and UPX baseline notes,
- decide whether to fork SMOL or start standalone.

Recommendation:

```
Start standalone, with SMOL as reference.
```

Reason:

A standalone prototype avoids early architectural friction. Later it can become a SMOL backend if that proves sensible.

Phase 1: RISC-V profile and relocation survey

Duration: 4-7 days.

Tasks:

- compile tiny C and assembly test objects,
- inspect relocations,
- document compiler flags and relocation output,
- identify minimal supported relocation set,
- test `-mrelax` and `-mno-relax`,
- test `-fno-plt` if relevant,
- test direct assembly `call symbol`,
- test toolchain support for `-march=rva23u64`,
- generate explicit fallback ISA strings for toolchains that do not accept profile names,
- compare `rv64gc`, `rv64gcv+b`, and `rva23u64` codegen,
- inspect ELF attributes and RISC-V flags emitted for each target.

Deliverable:

```
docs/riscv-relocation-survey.md
```

Exit criterion:

```
We know exactly which relocations are needed for the first dynamic call demo.
```

Phase 2: Minimal ELF64/RISC-V executable

Duration: 1 week.

Tasks:

- create handmade ELF header,
- create program headers,
- set correct RISC-V flags,
- create `_smolr_start`,
- create a no-import executable that exits or traps,
- run under qemu-riscv64,
- run on real hardware if available.

Exit criterion:

```
A handmade minimal RV64GC ELF starts correctly.
```

Phase 3: One imported function

Duration: 1-2 weeks.

Tasks:

- emit `DT_NEEDED` for libc,
- locate link map or dynamic loader structures,
- locate libc symbol table,
- resolve one symbol by hash or name,
- write address into import slot,
- implement one import stub,
- call `puts` or `write`.

Exit criterion:

```
SMOLR-built executable prints text using one dynamically resolved libc
symbol.
```

This is the first true proof of life.

Phase 4: Multiple imports and libraries

Duration: 1-2 weeks.

Tasks:

- multiple imported functions,
- multiple `DT_NEEDED` entries,
- deterministic library order,
- import table terminator strategy,
- error handling for unresolved symbols,
- libm test,
- basic SDL/Cairo smoke test.

Exit criterion:

```
SMOLR resolves multiple symbols across multiple shared libraries.
```

Phase 5: SMOLR frontend

Duration: 1-2 weeks.

Tasks:

- parse input object symbols,
- parse relocations,

- find libraries,
- map imports to libraries,
- generate runtime assembly,
- invoke assembler/linker,
- produce final executable,
- emit size report.

Exit criterion:

```
User can run one command to produce a SMOLR executable from demo.o.
```

Phase 6: RVA23 size/performance optimization pass

Duration: open-ended, first pass 1-2 weeks.

Tasks:

- rewrite resolver for RVC/Zcb density,
- tune register choices,
- shrink import table,
- compare hash strategies,
- compare SYSV/GNU hash lookup methods,
- test linker relaxation outcomes,
- remove redundant dynamic entries,
- compare single RWX vs split RX/RW,
- compare with UPX,
- write scalar bitmanip variants of hash/scan/decompressor kernels,
- write RVV variants of data-parallel kernels,
- benchmark vector-length-agnostic RVV code,
- compare `rv64gc`, `rv64gcv+b`, and `rva23u64` output.

Exit criterion:

```
SMOLR beats normal ld + UPX on the benchmark suite.
```

Phase 7: Demo-library stress tests

Duration: 1-2 weeks.

Tasks:

- Cairo A8 surface creation,
- Cairo text rendering,
- Pango layout,
- OpenGL/EGL minimal context,
- audio library call if needed,
- measure import counts and stub overhead.

Exit criterion:

```
SMOLR is proven useful for realistic ENO/SPINE demo scenarios.
```

Phase 8: Packer strategy

Duration: later.

Tasks:

- test UPX after SMOLR,
- inspect whether UPX harms or helps tiny SMOLR binaries,
- design RISC-V instruction filter if useful,
- evaluate custom decompressor,
- evaluate external decompression tricks where compo rules allow.

Exit criterion:

```
Final recommended compression path is known for 4k and 64k categories.
```

---

17. Suggested issue tracker

Initial issues:

1. Create cross-toolchain and qemu test harness.
2. Document RISC-V relocations emitted by GCC for simple external calls.
3. Generate minimal RV64GC ELF header accepted by Linux.
4. Determine required RISC-V `e_flags` for RV64GC/LP64D.
5. Implement one-symbol runtime resolver prototype.
6. Implement RISC-V import stub prototype.
7. Print hello through dynamically resolved libc symbol.
8. Add multiple imported symbols.
9. Add multiple libraries.
10. Add SMOLR frontend scanner for object relocations.
11. Add deterministic size report.
12. Add UPX comparison target.
13. Add Cairo smoke test.
14. Add RVC optimization pass.
15. Add linker relaxation audit.

---

18. Team roles

Toolchain lead

Ideal profile:

- GCC/binutils/ELF/ABI experience,
- comfortable with `readelf` and `objdump`,
- understands linker relaxation and relocation records,
- can debug dynamic loader weirdness.

This person owns:

- relocation survey,
- ELF validity,
- linker behavior,
- architecture decisions.

Runtime assembly lead

Ideal profile:

- strong RISC-V assembly,
- comfortable with ABI rules,
- can optimize for RVC,
- not afraid of startup code.

This person owns:

- `_smolr_start`,
- import resolver,
- import stubs,
- size optimization.

Test/build lead

Ideal profile:

- good with Make/CMake/Python/shell,
- qemu/cross-toolchain setup,
- reproducible benchmarks,
- CI discipline.

This person owns:

- test harness,
- size reports,
- benchmark matrix,
- regression tests.

A single heroic cave goblin can cover the first two roles. The third role should still be automated early, because byte-count regressions breed in dark corners.

---

19. Risk register

Risk: Dynamic loader internals differ or change

Mitigation:

- start with robust resolver strategy,
- document glibc version assumptions,
- test on multiple distros if possible,
- keep safer and smaller resolver modes separate.

Risk: RISC-V relaxation breaks hand assumptions

Mitigation:

- preserve relocation records until linker stage,
- test with `-mrelax` and `-mno-relax`,
- inspect final disassembly,
- avoid depending on temporary register side effects after relaxable sequences.

Risk: UPX already good enough for 64k

Mitigation:

- benchmark early,
- focus SMOLR on 4k or import-heavy cases where metadata overhead dominates,
- keep UPX as post-processing option.

Risk: Cairo/Pango imports too many symbols

Mitigation:

- measure import count early,
- provide direct Cairo-only mode,
- consider text rendering as build-time asset where possible,
- cache generated A8 surfaces at runtime.

Risk: Tool becomes too general

Mitigation:

- reject features aggressively,
- optimize for controlled demo code,
- keep non-goals visible in README.

20. First heroic task package

Give this to the toolchain expert:

Objective

Produce a feasibility report and proof-of-concept for a minimal dynamically linked **RVA23U64-first** Linux ELF executable using SMOLR principles.

The proof of concept may start with RV64GC for bootstrapping, but the report must answer how the design changes when the real target is RVA23U64 with RVV and bitmanip.

Required outputs

1. `riscv-profile-and-relocation-survey.md`
2. `hello-minimal-rv64gc.S`
3. `hello-minimal-rva23u64.S`
4. `hello-import-rv64gc.S`
5. `hello-import-rva23u64.S`
6. `Makefile`
7. `size-report.txt`

Success criteria

```
make run-qemu
```

prints something using a dynamically resolved libc function.

Stretch criteria

- Works on real RISC-V Ubuntu.
- Uses RVC/Zcb where possible.
- Shows size comparison against normal GNU ld and UPX.
- Shows comparison between RV64GC, RV64GCV+B fallback, and RVA23U64.
- Includes at least one tiny scalar bitmanip kernel experiment.
- Includes at least one tiny RVV kernel experiment if feasible.
- Documents which parts should become SMOLR proper.

21. Early commands

Example setup commands:

```
sudo apt install
  gcc-riscv64-linux-gnu
  binutils-riscv64-linux-gnu
  qemu-user
  qemu-user-binfmt
  make
  python3
```

Example inspection commands:

```
riscv64-linux-gnu-readelf -h -l -d -r -sW file.elf
riscv64-linux-gnu-objdump -dr file.elf
wc -c file.elf
qemu-riscv64 -L /usr/riscv64-linux-gnu ./file.elf
```

Example compilation command:

```
riscv64-linux-gnu-gcc
  -march=rv64gc
  -mabi=lp64d
  -Os
  -fno-stack-protector
  -fno-unwind-tables
  -fno-asynchronous-unwind-tables
  -nostartfiles
  -c hello.c -o hello-rv64gc.o

# Preferred when supported by the installed toolchain:
riscv64-linux-gnu-gcc
  -march=rva23u64
  -mabi=lp64d
  -Os
  -fno-stack-protector
  -fno-unwind-tables
  -fno-asynchronous-unwind-tables
  -nostartfiles
  -c hello.c -o hello-rva23u64.o
```

---

22. Final strategic recommendation

SMOLR should not begin as “port SMOL to RISC-V.”

It should begin as:

> A RISC-V-native proof-of-concept that borrows SMOL's central idea: replace bulky dynamic-linking metadata with a tiny runtime resolver and compact import table.

Then decide:

1. upstream into SMOL,
2. remain a separate SMOLR project,
3. become a backend shared with SMOL,
4. integrate with UPX/custom packer later.

The project wins if it produces measured results. The first real victory is not theoretical elegance. The first real victory is a tiny RV64GC ELF that prints one line through a dynamically resolved import and makes `wc -c` look confused.